Extreme Programming: Agile Framework for High-Quality Software Development

Cheryl Cheong Kah Voon[1], Iman Abadi Bin Mohd Nizwan[2]

Faculty of Computing, University Technology Malaysia,

81310 UTM Johor Bahru, Johor,Malaysia

imanabadi@graduate.utm.my

cherylcheongkah@graduate.utm.my

***Abstract*—Extreme Programming (XP) is an agile software development methodology introduced by Kent Back in 1996 during his work on the Chrysler C3 payroll project. A framework aimed at producing higher-quality software while also improving the development team's quality of life. XP is the most detailed agile framework in terms of effective software development practices. XP approach practices involve paired programming, test-driven development and continuous integration. This paper has provided a detailed explanation of XP and compared it with other Agile methodologies such as Scrum and Kanban. It goes on to discuss the advantages and disadvantages of XP and highlights its practical use in modern software development situations.**

Keywords—Extreme Programming, Agile Methodology, Software Development, Paired Programming, Test-Driven Development

I. INTRODUCTION

Agile system development technique is one of the most widely used methodologies in the world. Because of the increased need for several distinct systems from various industries such as banking, IT, business, fashion, biotechnology, and construction, systems must be built at a rapid speed while maintaining quality requirements[8]. As a result, the agile technique was adopted. Agile is a project management methodology that focuses on incremental and iterative stages to complete projects. The strategy includes continual feedback, which allows team members to adapt to new issues and stakeholders to communicate consistently[8]. Extreme programming is an Agile project management technique that emphasizes speed and simplicity, with short development cycles and minimal documentation[9]. XP differs from other agile methodologies in that it is considerably more disciplined, relying on frequent code reviews and unit testing to implement changes quickly. It is also highly creative and collaborative, emphasizing partnership throughout the creation process[9].

II. EXTREME PROGRAMMING PRACTICES

A. Paired Programming

Paired programming is the XP software formed by two programmers, sitting side by side on the same machine. It does mean that one programmer is writing the code and the other is watching. It is one person will write the codes and the other will review the code and think so that the two minds can come up with a better solution. Pair programming is a conversation between two programmers trying to simultaneously program and learn how to code better [10]. The benefits of paired programming are reducing the likelihood of errors, improving product quality and faster knowledge sharing. Additionally, paired programming promotes problem-solving skills and collaboration between two developers, which can lead to solving the problem easily.

B. Test-Driven Development(TDD)

The incremental method of software development is called test-driven development. It is based on automated unit tests, written and made to work in a concise cycle. TDD can ensure the code meets the specific requirements and enhances the code's quality. Writing the test before coding will push the developers to think critically about the software. Additionally, TDD encourages modular code since it is easier to understand, debug, and maintain [2]. It ensures developers can catch errors the first time and debug them during the debugging process by writing tests and code together, as it ensures developers can catch errors the first time.

C. Small Release

The small release is one of the practices for extreme programming. As the requirements in software development change rapidly so implementing small releases is useful from the perspective of the customer[1]. The small release is a simple system that is "productionized" quickly at least once every two to three months. Versions are then released daily, otherwise monthly [3].It will release several versions of the software to the customer for testing and checking before releasing it to the public. This step can reduce the risk of project failures and allow the developer to change based on the customer's review.

D. Continuous Integration(CI)

Continuous integration is the process of integrating new code into the code base as soon as it is available. The developer has the responsibility to run their code numerous times every day in order to ensure that the version is always functional. As a result, integrating and building the system several times each day could help in functionality regression and requirement changes[1]. All tests are executed and they have to be passed for the changes in the code to be accepted [3]. The good practice for continuous is having a dedicated computer where programmers can be involved in the code implementation and change the code if it doesn't work.

E. Simple Design

Simple design recognizes that the requirements for software development are constantly changing. It emphasizes creating the simplest solution that is currently implementable, with unnecessary code removed immediately[1]. XP consistently supports simple design and comprehensible code. It encourages the use of minimum classes, functions, or methods as feasible in order to make code easier to understand with as little documentation as possible outside of the source code. This may significantly reduce costs, and the developer can easily write and comprehend the code without having reviewed any documentation [1].

III. Comparison between other agile methodologies

A. XP vs Scrum

Scrum, like extreme programming, is an agile methodology. It is the most common agile software development model. It provides an iterative and incremental software development approach based on Japanese industry best practices. In Scrum, each product is launched based on customer needs, time constraints, competition, product quality, and available resources. XP contains twelve principles that provide specific instructions throughout the development process, whereas Scrum allows team members to choose their development methods while offering a framework rather than specific development techniques [5]. Furthermore, scrum deals with organizational challenges, whereas XP often concentrates on the engineering components of software projects, such as pair programming. Scrum is often used in larger teams while XP is more suitable for small or multi-functional teams.

B. XP vs Kanban

Kanban is a visual workflow and the work is broken into small and it will be written on a card and put on the board. The board has different columns and work progress. The card will move accordingly based on progress. Kanban is to eliminate problems that cause us to take longer time to fulfill the requirement not to skip the important step in the process while XP is to organize people to generate higher quality software more efficiently. Kanban is often chosen by the management and the value and goals are communicated down to the developer but XP is more suitable for a small team or multi-functional team.

IV. Advantage of the Extreme Programming(XP)

One of the main advantages of XP is development that centres around program code with continuous revision [6]. XP practices pair programming which requires two programmers to solve a coding problem by sharing ideas and working with collaboration to finish their task [7]. This fosters team richness and teamwork in prioritising continuous improvements which can lead to fixing issues such as bugs and logical errors present in the code of a developed application. XP is also known to revolve or centre around customers. The customers-centric approach allows the customer to be essentially involved with the development team and be considered as team members [7]. This makes the testing phase important and should be performed in all development stages before the implementation stage. The constant need for communication in XP and rigorous testing makes it an ideal methodology for developing a quality system that is up to standard. Aside from that, one of XP's five goals is simplicity, which aims to make software development as simple as possible and avoid dealing with functionalities that are not currently relevant but may be utilized in the future [6]. XP is against creating a more robust architecture than is necessary for the time being in order to avoid wasting time and energy developing functionalities that may or may not be used in the future. The reason for this is that frequent confusing customer requirements have shown to be pointless or unusable for a certain group of users [6].

V. Disadvantage of the Extreme Programming

Although XP has extensive benefits, the drawbacks are also to be noted. Due to XP being heavily revolved around teamwork, this can also be a disadvantage. For example, XP is less effective if it is done with outsourced teams [7]. This is because XP needs very skilled members who can collaborate, trust, and respect each other as well as being very self-organized. These skills are hard to find amongst team members who are unfamiliar with each other and prefer to work individually and solely focus on completing the project. This limits XP practices like pair programming. Team collaboration is also reduced if location constraints pose an issue because XP works well when interactions between team members and customers are done in person. Other than that, XP also suffers from weak documentation and lacks system design [7]. Documentation is crucial in system development as it reduces the recurrence of errors. Due to XP's nature of continuous change, proper documentation cannot be done easily, hence, identifying failures is challenging and is often high in risk. XP lacks system design due to it being code-centric. Although code implementation is important, prioritizing system design that fulfils requirements is more important to take into consideration than heavy code implementation as it affects the software's specifications of whether it satisfies the system requirements. Furthermore, XP is not appropriate for medium- to large-sized projects since extensive testing and code reorganization are performed sequentially for each iteration, reducing agility and increasing response time [7].

VI. Conclusion

Extreme programming is a methodology that is highly effective for teams who want to produce high-quality software with rapid change. It encourages collaboration, which centralizes the customer and focuses on adaptability and code quality. XP also emphasizes practices such as paired programming, test-driven development (TDD), continuous integration (CI), and small releases to lower the risk of products that do not meet the customer's requirement. Thus, XP is more suitable for small teams or multi-functional teams because it has more face-to-face communication and close collaboration with the teams. XP has extensive benefits, but it also has limitations. For instance, XP is plagued by inadequate documentation and a lack of system design. This limitation will cause XP not to be suitable for high medium to large-sized projects. Considering these limitations, XP provides reliable software that fulfils customer requirements. This benefit makes XP an effective approach to agile development situations. XP could improve on the part of documentation techniques in the future, seeking ways

to apply its concepts to larger projects or distributed teams, which may improve its applicability and effectiveness in a variety of project settings.

ACKNOWLEDGEMENTS

This paper and the research behind it would not have been possible without the exceptional support of our lecturer, Dr Aryati Binti Bakri. Her enthusiasm, knowledge and supervising of our detail or backlog have been an inspiration and kept our work on track about extreme programming (XP) to the final draft of this paper.

REFERENCES

[1] Shrivastava, A., Jaggi, I., Katoch, N., Gupta, D., & Gupta, S. (2021, July). A systematic review on extreme programming. In Journal of Physics: Conference Series (Vol. 1969, No. 1, p. 012046). IOP Publishing.B. Rieder, Engines of Order: A Mechanology of Algorithmic Techniques. Amsterdam, Netherlands: Amsterdam Univ. Press, 2020.

[2] Pančur, M., & Ciglarič, M. (2011). Impact of test-driven development on productivity, code and tests: A controlled experiment. *Information and Software Technology*, *53*(6), 557-573.

[3] Abrahamsson, P., Salo, O., Ronkainen, J., & Warsta, J. (2017). Agile software development methods: Review and analysis. *arXiv preprint arXiv:1709.08439*.

[4] Taibi, D., Lenarduzzi, V., Janes, A., Liukkunen, K., & Ahmad, M. O. (2017, April). Comparing requirements decomposition within the scrum, scrum with kanban, XP, and banana development processes. In *International Conference on Agile Software Development* (pp. 68-83). Cham: Springer International Publishing.

[5] Anwer, F., Aftab, S. S. S. M., Shah, S. S. M., & Waheed, U. (2017). Comparative analysis of two popular agile process models: extreme programming and scrum. International Journal of Computer Science and Telecommunications, 8(2), 1-7.

[6] Rostislav Fojtik, Extreme Programming in development of specific software, Procedia Computer Science, Volume 3, 2011, Pages 1464-1468, ISSN 1877-0509, https://doi.org/10.1016/j.procs.2011.01.032.

[7] Qureshi, M. R. J., & Ikram, J. S. (2015). Proposal of the enhanced extreme programming model. International Journal of Information Engineering and Electronic Business, 7(1), 37. https://www.academia.edu/download/48864398/IJIEEB-V7-N1-5.pdf

[8] Staff, C. (2024, April 17). What is agile? and when to use it. Coursera. https://www.coursera.org/articles/what-is-agile-a-beginners-guide

[9] Raeburn, A. (2024, February 13). What is Extreme Programming (XP)? [2024] • Asana. Asana. https://asana.com/resources/extreme-programming-xp

[10] Beck, K. (2000). *Extreme programming explained: embrace change*. Addison-Wesley.